Jac for Database Programming

Existing language work, compiler mechanisms and historical evidence | September 4–5, 2026

Research objective

Make persistent graph applications easier to write, while the compiler and runtime organize efficient database access.

Two linked questions: how much database plumbing does the programmer write, and how effectively does the system execute that program?

Sources: DBHarness historical artifacts; exact paths and SHA-256 values accompany this deck.

Research problem: one application, two kinds of work

Expressing business logic and organizing database execution both consume development effort.

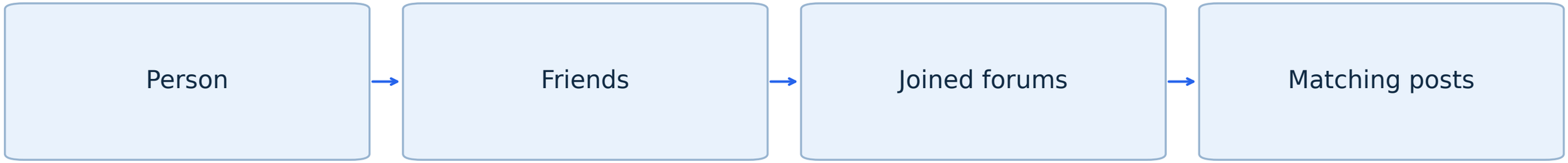
Application logic	Database execution decisions
Follow domain relationships	Choose joins and query boundaries
Filter, count, rank and update	Batch loads and avoid per-object round trips
Enforce application behavior	Project fields, load objects and maintain access checks

The opportunity: express domain intent once and let the language implementation handle more execution decisions.

Source: session0904/BURDEN.md; session0904/ic5/COMPILER_PLAN.md

Example problem: find the most active relevant forums

IC5 benchmark: one-hop friends, forums joined after a date, and posts by qualifying friends.



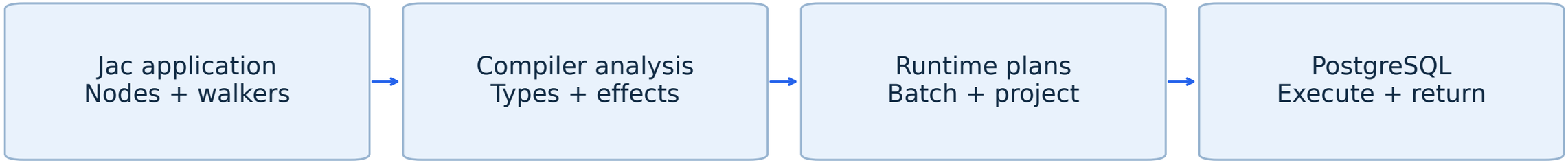
Count per forum → order by count and forum ID → return the top 20

The relevant relationships are explicit; the program must combine traversal, edge filtering, membership, aggregation and ranking.

Source: session0904/ic5/COMPILER_PLAN.md; benchmark/snb/jaseci-pg/app.jac

Solution: keep graph intent visible to the system

Jac supplies typed nodes, edges and walkers; compiler/runtime mechanisms organize their database execution.



Graph traversal and application logic share the same language

Existing mechanisms include typed lookup, ranged hops, loop prefetch, nested-count lowering and delayed hydration.

Source: session0904/ic5/build/REPORT.md; historical Jac compiler/runtime commits

A concrete language example: typed, bounded traversal

Source excerpt: select people within three Knows hops and filter by firstName.

```
start = jobj(self.person_jid, Person);

for (f, dist) in [start<-:Knows:*3:->[?:Person,
    firstName==self.first_name]] {
    ...
}
```

The typed lookup replaced nine explicit type-check blocks in the SNB burden copy. Ranged hops replaced closure calls plus hydration/filtering code. Ellipsis abbreviates the original loop body.

Source: benchmark/snb/jaseci-pg/app.jac:284,294; session0904/BURDEN.md

What the colleague implemented

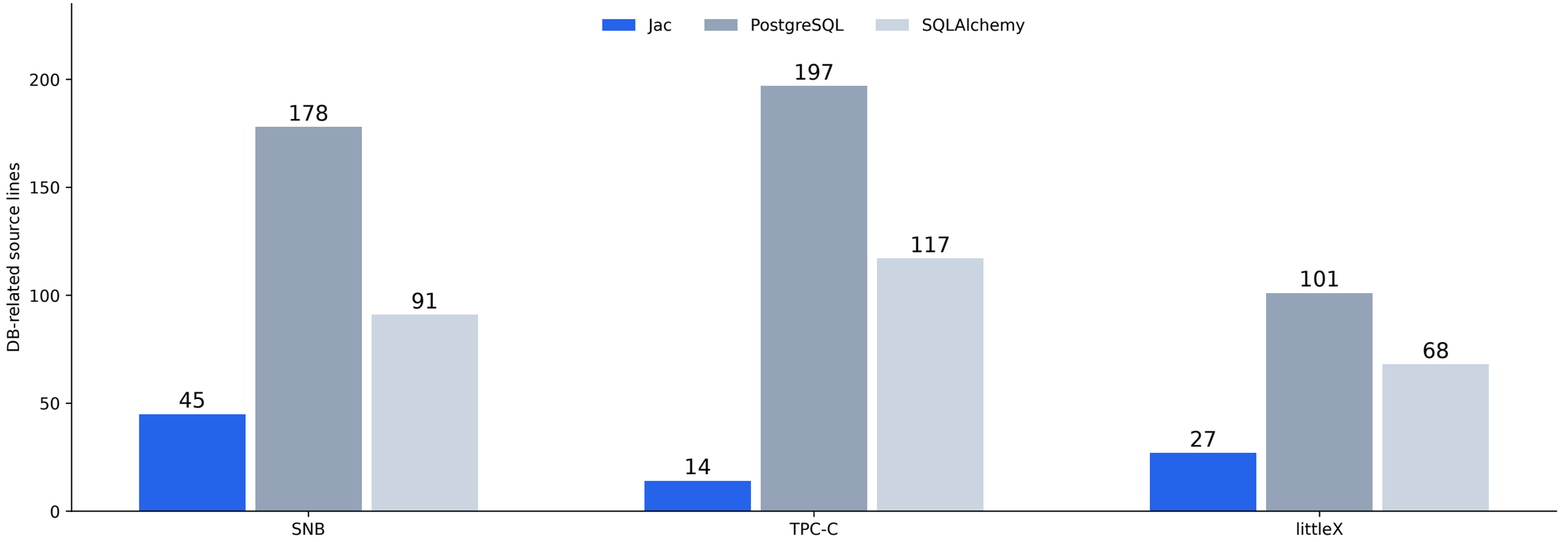
The existing contribution spans language expression, execution mechanisms and application evidence.

Layer	Existing work
Language / API	Typed jobj lookup; variable-length graph hops; field and edge predicates
Compiler	Loop prefetch; field-use projection; nested-hop count recognition; sort/slice fusion
Runtime / storage	Batched adjacency; SQL aggregation and Top-K; typed projections; request-local memoization
Applications / evaluation	SNB, TPC-C and littleX; SQL / ORM / Neo4j counterparts; parity gates and burden counts

Source: session0904/BURDEN.md; session0904/ic5/build/REPORT.md; historical runtime commit history

Data: fewer database-related lines

Historical burden copies, using the colleague’s database-access line classification.



Database-related lines are classified source lines. The counts describe these implementations; development effort also depends on the tasks programmers perform.

Source: benchmark/burden/viewer_counts.csv; FORMAT_FAIRNESS.md; session0904/BURDEN.md

Data: total application size has a different ranking

Historical LoC / database-related LoC, including all available comparison arms.

Application	Jac	PostgreSQL	SQLAlchemy	Neo4j
SNB	320 / 45	315 / 178	330 / 91	202 / 71
TPC-C	225 / 14	302 / 197	232 / 117	—
littleX	174 / 27	250 / 101	281 / 68	229 / 75

Jac has fewer DB-related lines in these comparisons. SNB total LoC is 320 for Jac, 315 for PostgreSQL and 202 for Neo4j. Both measurements belong in the evaluation.

Source: benchmark/burden/viewer_counts.csv; FORMAT_FAIRNESS.md

Solution example: optimize an unchanged IC5 walker

Historical compiler/runtime ladder: baseline c12ba889d → final 32c5d1dbb.

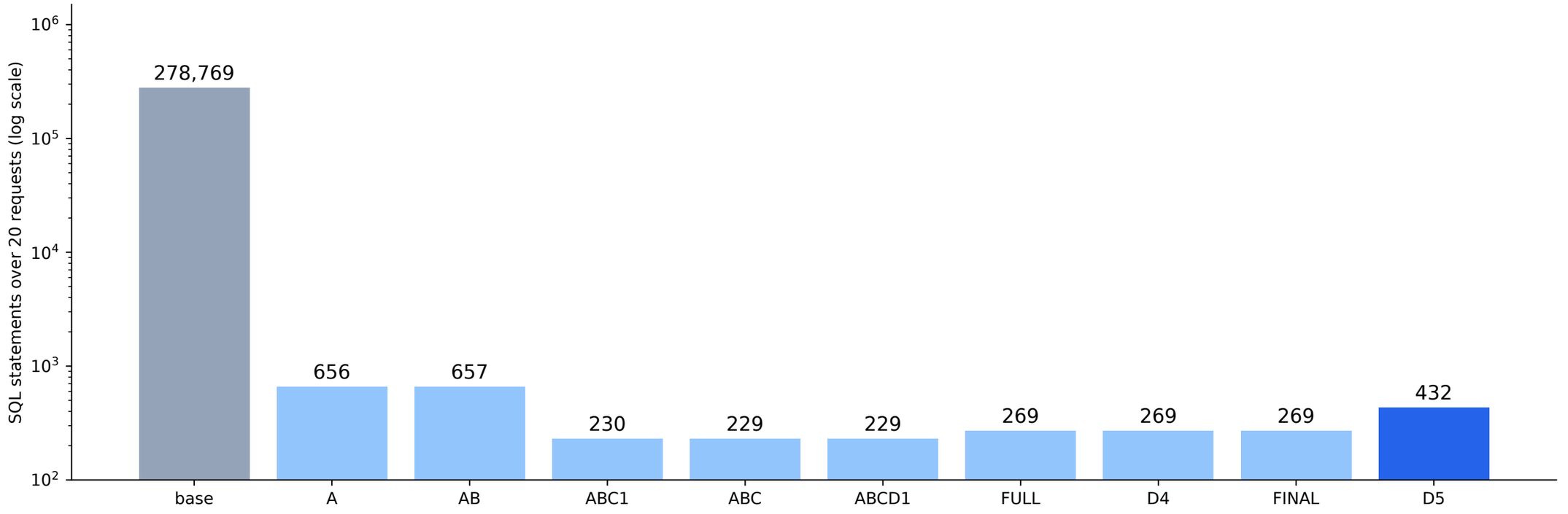
Program structure	System transformation
Nested per-object graph access	Batch each frontier; reuse request-local adjacency results
Count posts whose author is in a qualifying set	Lower the recognized nested count to SQL aggregation
Sort counts and take 20	Perform ordering and Top-K inside SQL
Read winning forum fields	Project needed fields and hydrate selected winners

The walker still constructs qualifying forum/member pairs. The final execution combines application-side work with generated database operations.

Source: session0904/ic5/build/REPORT.md; COMPILER_PLAN.md

Data: IC5 SQL work falls by roughly 645×

20 historical parameter rows; identical walker; all rows pass output parity at every rung.



Baseline 278,769 → D5 432 statements (645.3× fewer). Intermediate counts reflect different hydration and planning choices.

Source: session0904/ic5/build/gates/ic5_*.json; values recomputed from the raw arrays

Data: the historical IC5 optimization ladder

Application-request timing, milliseconds; each rung has a paired hand-written PostgreSQL measurement.

Rung	SQL total	Jac p50 ms	Paired PG p50 ms	Output parity
base	278,769	7542.00	80.35	20 / 20
A	656	191.35	84.30	20 / 20
AB	657	120.65	64.95	20 / 20
ABC1	230	103.45	60.65	20 / 20
ABC	229	104.15	60.05	20 / 20
ABCD1	229	126.90	82.60	20 / 20
FULL	269	69.25	51.10	20 / 20
D4	269	78.95	63.90	20 / 20
FINAL	269	109.10	75.85	20 / 20
D5	432	75.50	60.05	20 / 20

September 5 ran on a busy host; timing windows varied across rungs. Source-pinned SQL counts and result parity provide the clearest evidence of the transformation.

Source: session0904/ic5/build/gates/ic5_*.json; arithmetic medians over 20 rows

Data: SNB performance depends on the query

September 4 client-request p50, milliseconds; historical CONTROL / BEST flag configurations.

Query	PG	SQLAlchemy	Neo4j	Jac CONTROL	Jac BEST
IS1	0.91	1.30	1.47	2.66	2.44
IS2	9.95	7.91	2.10	5.30	4.36
IS3	1.38	1.92	1.93	4.06	3.70
IC1	28.33	83.62	7.42	5.84	5.67
IC2	42.79	41.33	15.28	9.94	8.53
IC5	73.48	61.24	17.09	4632.08	4567.20
IC13	23.15	47.43	2.60	22.57	21.53

Jac is faster than the PostgreSQL implementation for IC1 and IC2 in this cohort. The large IC5 gap motivated the later compiler ladder. Existing application strategies include batching and explicit query helpers.

Source: session0904/run/data/snb/snb_master.csv; this cohort predates the September 5 IC5 ladder

Data: TPC-C places Jac between or near SQL and ORM

September 4 client-request p50, milliseconds; one warehouse, sequential transaction schedule.

Operation	PostgreSQL	SQLAlchemy	Jac CONTROL	Jac BEST
New-Order	9.97	23.30	14.05	13.78
Payment	7.98	10.66	10.82	10.65

The measured 800 requests include 437 successful New-Orders, 357 Payments and 6 expected rollbacks. The CSV groups those rollbacks with New-Order. Timing covers the application request.

Source: session0904/run/data/tpcc/tpcc_master.csv; 100 warmup + 800 measured requests

Data: full-feed cost grows with tweets per user

September 4 application-server p50, milliseconds; 100 users and 50 follows per user.

Tweets / user	PostgreSQL	SQLAlchemy	Neo4j	Jac CONTROL	Jac BEST
t20	4.14	9.84	26.95	12.99	16.44
t50	10.64	24.35	67.25	22.60	23.39
t100	21.10	50.85	137.26	41.92	43.11
t200	42.39	122.20	282.67	84.14	85.49

Each full feed returns 1,000 / 2,500 / 5,000 / 10,000 tweets respectively. The PostgreSQL implementation is faster for this workload; the historical study also includes writes and paged-feed mixes.

Source: session0904/run/data/littlex_{baselines,control,best}/w1_master.csv

What the historical evidence establishes

Programming abstraction and execution quality are two dimensions of the same research objective.

Finding	Evidence
Less database-specific application code	Lower DB-related line counts across the three application families
System improvements can preserve application source	IC5 unchanged walker: 278,769 → 432 SQL statements; 20/20 output parity
Execution quality depends on workload and program shape	SNB IC1/IC2 advantages; TPC-C and full-feed comparisons show remaining costs
The existing system exposes useful optimization information	Typed relations, traversal direction, predicates, field use and recognized loop effects

Sources: historical burden metrics, IC5 raw parity/count gates and SNB/TPC-C/littleX master CSVs

Research direction: make efficient DB programming easier

Evaluate the language and system together through concrete application tasks.

Programmer task: implement and then modify a graph-backed feature.

Measure the work: database-specific code, manual batching, query helpers, and edits required when the feature changes.

Measure the execution: correctness, SQL counts and clearly defined latency, across natural program variants.

Goal: keep application intent clear while the system carries more of the database execution burden.

Proposal derived from the historical evidence; task and maintenance measurements are the next evaluation step.